# Requirements Gathering

StockWise AI – Retail Demand Forecasting & Sales Prediction

DEPI Machine Learning Capstone · Team of 5 · 31 October 2026

This document describes who the system is for, what they need from it, and the functional and non-functional requirements the team will build and test against. It covers stakeholder analysis, user stories and use cases, functional requirements, non-functional requirements, assumptions and constraints, and a traceability table linking the official Project 2 requirements to our requirement IDs.

**Priorities (MoSCoW).** Must = Level 1 core (graded). Should = Level 2 inventory layer. Could = Level 3 dashboard and AI assistant, and Level 4 cloud deployment. Won't = out of scope for this project. Should and Could items are planned extensions, built only as time allows after Level 1 is complete.

## 1. Stakeholder Analysis

StockWise AI is an academic project built on public Favorita data, so the business stakeholders below are assumed roles that represent who would use such a system in a real supermarket chain. The instructor, the DEPI evaluators and the team are the real stakeholders of the project.

| Stakeholder | Interest in the system | Main needs | Influence | Type |
|---|---|---|---|---|
| **Store / inventory manager** | Knowing how much of each product family will sell in the coming days | Clear 16-day forecasts per store and family; simple reorder advice; plain-language answers | Medium | Assumed (end user) |
| **Purchasing / supply-chain planner** | Planning orders and supplier lead times | Forecast accuracy figures; safety stock and reorder points; error patterns by store and family | Medium | Assumed (end user) |
| **Business management** | Fewer empty shelves and less wasted stock | Summary of model quality versus a simple baseline; trustworthy, explainable results | High | Assumed |
| **Instructor & DEPI evaluators** | Grading the capstone against the Project 2 scope | All required scope delivered; documentation; MLflow evidence; each member can explain their part | High | Real |
| **Project team (5 members)** | Building, testing and presenting the system | Clean data, a reproducible pipeline, shared code on GitHub, experiment history in MLflow | High | Real |
| **Dataset provider (Kaggle / Favorita)** | Correct use of the published data | Data used under Kaggle's terms; raw data not redistributed in our repository | Low | Real |

## 2. User Stories & Use Cases

### 2.1 User Stories

Each story follows the form “As a …, I want …, so that …” and has acceptance criteria that tell us when it is done.

| ID | User story | Acceptance criteria | Priority |
|---|---|---|---|
| **US-01** | As a store manager, I want a daily sales forecast for each product family for the next 16 days, so that I can plan orders. | Forecasts exist for every store × family for each of the 16 test days; no negative values. | Must |
| **US-02** | As a purchasing planner, I want to know how accurate the forecasts are, so that I know how much to trust them. | MAE, RMSE and WAPE reported for every model on the same test period. | Must |
| **US-03** | As business management, I want to see whether the models beat a simple guess, so that the effort is justified. | Every model is compared against the naive baseline in one table. | Must |
| **US-04** | As a planner, I want to see where the forecasts are most wrong, so that I can be careful with those products. | Residual analysis over time, by store, by family, and holidays vs normal days. | Must |
| **US-05** | As a store manager, I want to understand how promotions and holidays affect sales, so that I can prepare for them. | EDA charts and short written findings on promotion and holiday effects. | Must |
| **US-06** | As the instructor, I want to see predicted versus actual sales, so that I can judge model behaviour visually. | Predicted-vs-actual plots for the chosen model on the test period. | Must |
| **US-07** | As a team member, I want every experiment recorded, so that we can compare models and repeat the best one. | Every training run appears in MLflow with parameters, metrics and plots. | Must |
| **US-08** | As a new team member or evaluator, I want to rerun the whole pipeline from instructions, so that the results are reproducible. | Following the README alone reproduces the comparison table. | Must |
| **US-09** | As an inventory manager, I want a reorder / OK / overstock flag per store and family, so that I know what to order. | A flag and reorder point exist for every store × family (simulated stock data). | Should |
| **US-10** | As a store manager, I want a dashboard where I pick a store and family, so that I can see forecasts without code. | Dashboard shows forecasts and charts for any chosen store and family. | Could |
| **US-11** | As a store manager, I want to ask questions in plain language (e.g. “which stores may run out of dairy next week?”), so that I get answers quickly. | The assistant answers prepared test questions correctly using only our outputs. | Could |
| **US-12** | As management, I want the final ordering decision to stay with a person, so that mistakes are not automated. | The assistant only recommends; it cannot place or change orders. | Could |

### 2.2 Use Cases

The use cases below describe the main interactions. The use-case diagram is part of the System Analysis & Design documents (due 6 Nov).

| ID | Use case | Primary actor | Priority |
|---|---|---|---|
| **UC-01** | Train and evaluate forecasting models | Project team | Must |
| **UC-02** | Generate a 16-day sales forecast | Project team (batch run) | Must |

| ID | Use case | Primary actor | Priority |
|---|---|---|---|
| **UC-03** | Review model performance | Instructor / planner | Must |
| **UC-04** | Get reorder recommendations | Inventory manager | Should |
| **UC-05** | Ask the AI assistant a question | Store manager | Could |

| **UC-01** | **Train and evaluate forecasting models** |
|---|---|
| **Actor** | Project team |
| **Preconditions** | Raw Kaggle files are downloaded; cleaning and feature steps run without errors. |
| **Main flow** | 1. The team runs the pipeline, which loads, cleans and merges the data.<br>2. Features are built (calendar, lags ≥ 16 days, shifted rolling statistics, promotion, holiday, store and family).<br>3. The data is split by date into train, validation and test periods.<br>4. Each model (naive baseline, Linear, Polynomial, Random Forest, XGBoost) is trained on the training period.<br>5. Each model is scored with MAE, RMSE and WAPE, and the run is logged in MLflow. |
| **Alternative flow** | If a model's scores look too good to be true, the team stops and checks the features for leakage before continuing. |
| **Postconditions** | Five runs exist in MLflow with comparable scores on the same test period. |

| **UC-02** | **Generate a 16-day sales forecast** |
|---|---|
| **Actor** | Project team (batch run) |
| **Preconditions** | A final model has been chosen and saved (UC-01). |
| **Main flow** | 1. The pipeline builds features for the 16 days to forecast.<br>2. The saved model predicts daily sales for every store × family.<br>3. Negative predictions are set to zero.<br>4. Forecasts are saved to a file (date, store, family, predicted sales). |
| **Alternative flow** | If input columns are missing or contain unexpected values, the run stops with a clear error message. |
| **Postconditions** | A forecast file is available for evaluation, the inventory layer and the dashboard. |

| **UC-03** | **Review model performance** |
|---|---|
| **Actor** | Instructor / purchasing planner |
| **Preconditions** | UC-01 is complete. |
| **Main flow** | 1. The actor opens the model comparison table and the MLflow experiment.<br>2. The actor compares every model against the naive baseline.<br>3. The actor reviews predicted-vs-actual plots and residual analysis by time, store, family and holidays. |
| **Postconditions** | The actor understands which model is best, by how much, and where it makes the largest errors. |

| UC-04 | Get reorder recommendations (as time allows) |
|---|---|
| **Actor** | Inventory manager |
| **Preconditions** | A forecast file exists (UC-02); simulated stock levels and lead times exist. |
| **Main flow** | 1. The system calculates safety stock and the reorder point for each store × family.<br>2. It compares current (simulated) stock with the reorder point.<br>3. It labels each series as reorder, OK or overstock and shows the suggested quantity. |
| **Postconditions** | A recommendation table is available; ordering decisions remain with the manager. |

| UC-05 | Ask the AI assistant a question (as time allows) |
|---|---|
| **Actor** | Store manager |
| **Preconditions** | The dashboard is running; forecast and recommendation files exist. |
| **Main flow** | 1. The manager types a question in plain language.<br>2. A ready-made LLM chooses a tool that reads our output files (for example, low-stock items for a store).<br>3. The assistant answers using only the data returned by the tool. |
| **Alternative flow** | If the data does not contain the answer, the assistant says so instead of guessing. |
| **Postconditions** | The manager gets an answer and a recommendation; no order is placed automatically. |

## 3. Functional Requirements

Each requirement has an ID so it can be traced to design, code and tests. Column names refer to the Kaggle files (train.csv, stores.csv, oil.csv, holidays_events.csv, transactions.csv).

### 3.1 Data Loading & Cleaning

| ID | The system shall… | Priority |
|---|---|---|
| **FR-01** | Load train.csv, stores.csv, oil.csv and holidays_events.csv and confirm their shapes, columns and date ranges. | Must |
| **FR-02** | Merge store information and daily oil prices into the sales data using store_nbr and date. | Must |
| **FR-03** | Fill missing oil prices (planned: carry the last known price forward) and document the method. | Must |
| **FR-04** | Treat holidays with transferred = True as normal days, and separate real holidays from other event rows. | Must |
| **FR-05** | Detect and handle outliers and long zero-sales series, with each decision documented. | Must |
| **FR-06** | Save the cleaned dataset to data/processed/ so later steps do not repeat cleaning. | Must |

### 3.2 Exploratory Data Analysis

| ID | The system shall… | Priority |
|---|---|---|
| **FR-07** | Plot total sales over time and show weekly and yearly seasonality. | Must |
| **FR-08** | Compare sales by product family and by store. | Must |
| **FR-09** | Show the effect of promotions, holidays and the April 2016 earthquake on sales. | Must |
| **FR-10** | Summarise the EDA findings in short written notes used by the feature engineering step. | Must |

### 3.3 Feature Engineering

| ID | The system shall… | Priority |
|---|---|---|
| **FR-11** | Create calendar features: day of week, day of month, month, year, weekend and payday flags. | Must |
| **FR-12** | Create lag features of at least 16 days, computed separately for each store × family. | Must |
| **FR-13** | Create rolling mean and standard deviation features, shifted by at least 16 days, per store × family. | Must |
| **FR-14** | Create promotion (onpromotion) and holiday / event features. | Must |
| **FR-15** | Encode store and product-family categories, fitting any encoder or scaler on the training period only. | Must |
| **FR-16** | Provide all feature steps as reusable functions in src/features.py. | Must |

## 3.4 Modeling

| ID | The system shall… | Priority |
|---|---|---|
| **FR-17** | Split the data by date: train up to 30 Jun 2017, validation 1–30 Jul 2017, test 31 Jul – 15 Aug 2017. | Must |
| **FR-18** | Provide a naive baseline forecast to compare all models against. | Must |
| **FR-19** | Train Linear Regression, Polynomial Regression, Random Forest and XGBoost models using random_state = 42. | Must |
| **FR-20** | Tune model settings using the validation period only; score the test period once for the final model. | Must |
| **FR-21** | Set negative sales predictions to zero. | Must |

## 3.5 Evaluation & Tracking

| ID | The system shall… | Priority |
|---|---|---|
| **FR-22** | Calculate MAE, RMSE and WAPE for every model on the same test period. | Must |
| **FR-23** | Produce a model comparison table including the naive baseline. | Must |
| **FR-24** | Analyse residuals over time, by store, by product family, and on holidays versus normal days. | Must |
| **FR-25** | Plot predicted versus actual sales and show feature importance for tree-based models. | Must |
| **FR-26** | Log every training run in MLflow with its parameters, metrics and plots. | Must |
| **FR-27** | Save forecasts to a file with date, store_nbr, family and predicted sales. | Must |
| **FR-28** | Provide a README with installation, data download and run instructions. | Must |

## 3.6 Extensions (as time allows)

| ID | The system shall… | Priority |
|---|---|---|
| **FR-29** | Simulate stock levels and supplier lead times for each store × family. | Should |
| **FR-30** | Calculate safety stock, reorder point and reorder quantity from forecasts and forecast errors. | Should |
| **FR-31** | Label each store × family as reorder, OK or overstock. | Should |
| **FR-32** | Provide a dashboard (planned: Streamlit) to view forecasts, model comparison and reorder flags by store and family. | Could |
| **FR-33** | Provide an AI assistant using a ready-made LLM that answers questions by calling tools that read our output files. | Could |
| **FR-34** | Deploy the forecasting pipeline or dashboard to the cloud (platform to be decided). | Could |

### 3.7 Out of Scope (Won't)

- **Deep learning models** (e.g. LSTM): not part of the agreed scope.
- **Training or fine-tuning an LLM:** the assistant uses a ready-made model only.
- **Automatic ordering:** the system recommends; people decide.
- **Real store systems:** no connection to live point-of-sale or stock systems; stock data is simulated.
- **Real-time forecasting:** forecasts are produced in batch runs.

## 4. Non-functional Requirements

Non-functional requirements describe how well the system must work. Each has a check the team can perform.

| ID | Category | Requirement | How we check it |
|---|---|---|---|
| **NFR-01** | Correctness | No data leakage: date-based split only, lags and rolling windows ≥ 16 days and shifted, transactions not used as a feature, encoders fitted on training data only. | Leakage checklist reviewed by Adham and Sara before final scoring |
| **NFR-02** | Reproducibility | The same code and data give the same results: fixed random_state = 42 and pinned library versions in requirements.txt. | Two runs produce identical metrics |
| **NFR-03** | Reproducibility | Someone who did not write the code can rerun the pipeline from the README. | Rerun by a different team member |
| **NFR-04** | Performance | The full pipeline runs within the memory and session limits of free Google Colab. | Completes without memory errors |
| **NFR-05** | Performance | If the dashboard is built, a forecast for one store and family is shown in under 5 seconds. | Timed during testing |
| **NFR-06** | Maintainability | Reusable code lives in src/ as small, commented functions with clear names; notebooks use small cells. | Code review by the review partner |
| **NFR-07** | Usability | Every chart has a title, axis labels and units; outputs are understandable by a non-technical manager. | Checked during report review |
| **NFR-08** | Explainability | Results are explained through feature importance, residual analysis and comparison with the baseline, not by a single score such as $R^2$. | Present in the evaluation report |
| **NFR-09** | Security | No API keys, passwords or tokens in GitHub; secrets are kept in a .env file listed in .gitignore. | Repository check before each merge |
| **NFR-10** | Security | The AI assistant's tools can only read our output files; they cannot change data or place orders. | Tool list reviewed; tested with misuse questions |
| **NFR-11** | Data & licensing | Raw Kaggle data is not committed to GitHub; each user downloads it from Kaggle. | data/raw/ listed in .gitignore |
| **NFR-12** | Reliability | The pipeline checks its inputs (files, columns, missing values) and stops with a clear error message when something is wrong. | Tested with a missing file and a missing column |
| **NFR-13** | Portability | The code runs on Python 3 in Google Colab and in local Jupyter. | Run in both environments |

| ID | Category | Requirement | How we check it |
|---|---|---|---|
| **NFR-14** | Collaboration | Work is version-controlled on GitHub with one branch per member and clear commit messages. | Commit history review |

## 5. Assumptions & Constraints

- **Data:** the Kaggle Favorita dataset (January 2013 – August 2017) is used as provided; no newer data is available.
- **Forecast horizon:** 16 days, matching the length of the test period.
- **Stock data:** real stock levels and supplier lead times are not available, so Level 2 uses simulated values.
- **Tools:** free tools only (Google Colab free tier, GitHub, MLflow); this limits memory and training time.
- **Team and time:** five members with their first ML project; first working version of Level 1 targeted for 6 November 2026, code freeze on 30 November 2026.
- **Users:** store managers, planners and management are assumed roles; the system is not used by a real company.

## 6. Traceability to the Project 2 Requirements

Every requirement in the official Project 2 description is covered by at least one functional requirement.

| Official Project 2 requirement | Covered by | Owner |
|---|---|---|
| Time-oriented exploratory analysis | FR-07 – FR-10 | Hazem |
| Handle missing values and outliers | FR-03, FR-05 | Zenab |
| Historical sales, lag variables, rolling statistics | FR-12, FR-13 | Adham |
| Product and store characteristics | FR-02, FR-15 | Zenab, Adham |
| Appropriate scaling and encoding | FR-15 | Adham |
| Linear Regression | FR-19 | Hazem |
| Polynomial Regression | FR-19 | Zenab |
| Ensemble / gradient boosting | FR-19 (Random Forest, XGBoost) | Adham, Sara |
| Evaluate prediction performance | FR-18, FR-22, FR-23 | Sara, Yussuf |
| Analyse residual / error patterns | FR-24 | Sara |
| Visualise predicted vs actual values | FR-25 | Sara |
| Track experiments with MLflow | FR-26 | Sara |
| Cleaned dataset | FR-06 | Zenab |
| Feature-engineering pipeline | FR-16 | Adham |
| README | FR-28 | Zenab |
| Optional: batch prediction deployment | FR-27, FR-34 | Yussuf |